

# CREATE SCHEMA AND MODELS

## Introduction

In the Online Complaint Registration System, MongoDB is used as the NoSQL database to store application data. Mongoose acts as an Object Data Modeling (ODM) library, allowing the application to define schemas and models for interacting with the database.

A **Schema** defines the structure of a document, while a **Model** is created from the schema and is used to perform Create, Read, Update, and Delete (CRUD) operations.

---

## 1. User Schema (User.js)

The User model stores information about registered users and administrators.

| Field     | Data Type | Description         |
|-----------|-----------|---------------------|
| name      | String    | User's full name    |
| email     | String    | User email (unique) |
| phone     | String    | Mobile number       |
| password  | String    | Encrypted password  |
| role      | String    | User/Admin          |
| createdAt | Date      | Registration date   |

### Sample code

```
const mongoose = require("mongoose");

const userSchema = new mongoose.Schema({
  name:{
    type:String,
```

```

        required:true
    },
    email:{
        type:String,
        required:true,
        unique:true
    },
    phone:{
        type:String,
        required:true
    },
    password:{
        type:String,
        required:true
    },
    role:{
        type:String,
        default:"User"
    }
}, {
    timestamps:true
});

module.exports = mongoose.model("User", userSchema);

```

## Complaint Schema (Complaint.js)

The Complaint model stores all complaints submitted by users.

| Field       | Data Type | Description        |
|-------------|-----------|--------------------|
| userId      | ObjectId  | Reference to User  |
| department  | String    | Department name    |
| category    | String    | Complaint category |
| title       | String    | Complaint title    |
| description | String    | Complaint details  |

|           |        |                                 |
|-----------|--------|---------------------------------|
| location  | String | Complaint location              |
| image     | String | Uploaded image                  |
| status    | String | Pending/In<br>Progress/Resolved |
| createdAt | Date   | Complaint date                  |

## Sample code

```
const mongoose = require("mongoose");

const complaintSchema = new mongoose.Schema({

  userId:{
    type:mongoose.Schema.Types.ObjectId,
    ref:"User",
    required:true
  },

  department:{
    type:String,
    required:true
  },

  category:{
    type:String,
    required:true
  },

  title:{
    type:String,
    required:true
  },

  description:{
    type:String,
    required:true
  },

  location:{
    type:String,
    required:true
  }
```

```
    },  
  
    image:{  
      type:String  
    },  
  
    status:{  
      type:String,  
      default:"Pending"  
    }  
  },  
  {  
    timestamps:true  
  }  
});  
  
module.exports = mongoose.model("Complaint", complaintSchema);
```

## Department Schema (Department.js)

Stores available government departments.

### Sample code

```
const mongoose = require("mongoose");  
  
const departmentSchema = new mongoose.Schema({  
  
  departmentName:{  
    type:String,  
    required:true  
  },  
  
  description:{  
    type:String  
  }  
},  
{  
  timestamps:true  
});
```

```
module.exports = mongoose.model("Department", departmentSchema);
```

## Notification Schema (Notification.js)

Stores notifications sent to users regarding complaint updates.

### Sample code

```
const mongoose = require("mongoose");

const notificationSchema = new mongoose.Schema({

  userId:{
    type:mongoose.Schema.Types.ObjectId,
    ref:"User"
  },

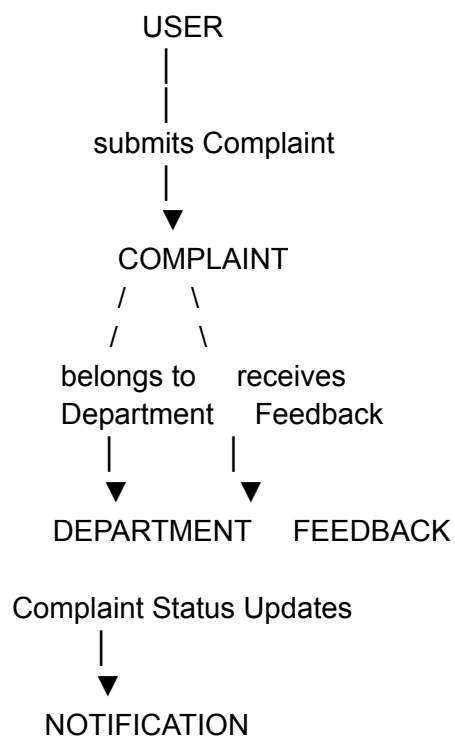
  message:{
    type:String,
    required:true
  },

  isRead:{
    type:Boolean,
    default:false
  }

},{
  timestamps:true
});

module.exports = mongoose.model("Notification", notificationSchema);
```

## Schema Relationship



## Summary

The Online Complaint Registration System uses **Mongoose schemas** to organize and validate application data. Each schema represents a specific entity such as **User**, **Complaint**, **Department**, **Feedback**, and **Notification**. These models ensure data consistency, simplify CRUD operations, and establish relationships between collections using MongoDB references. This modular structure makes the application scalable, maintainable, and easy to extend with additional features in the future.